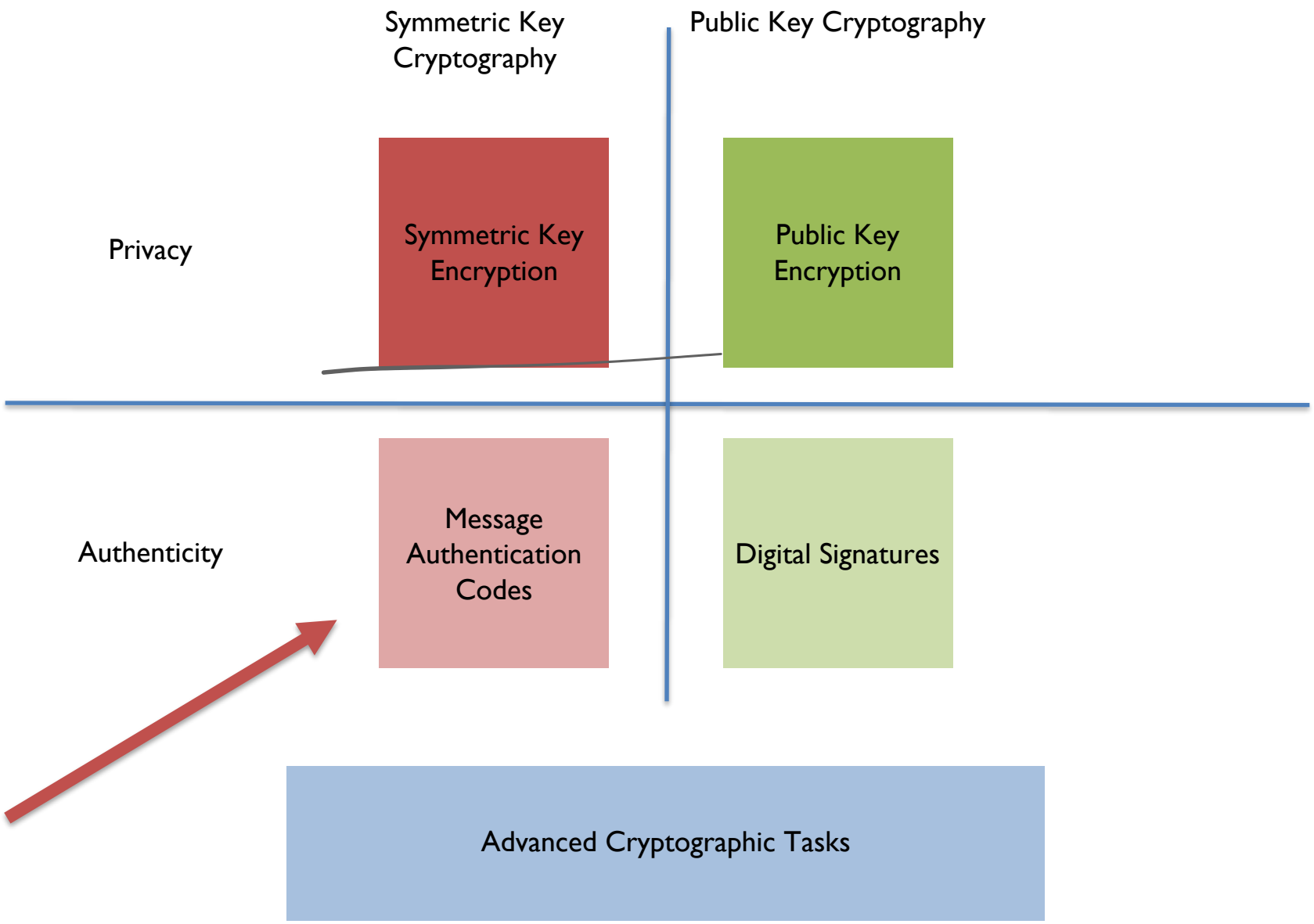
CISPA
HELMHOLTZ CENTER FOR INFORMATION SECURITY

Cryptography

Lecture 08: Hash Functions

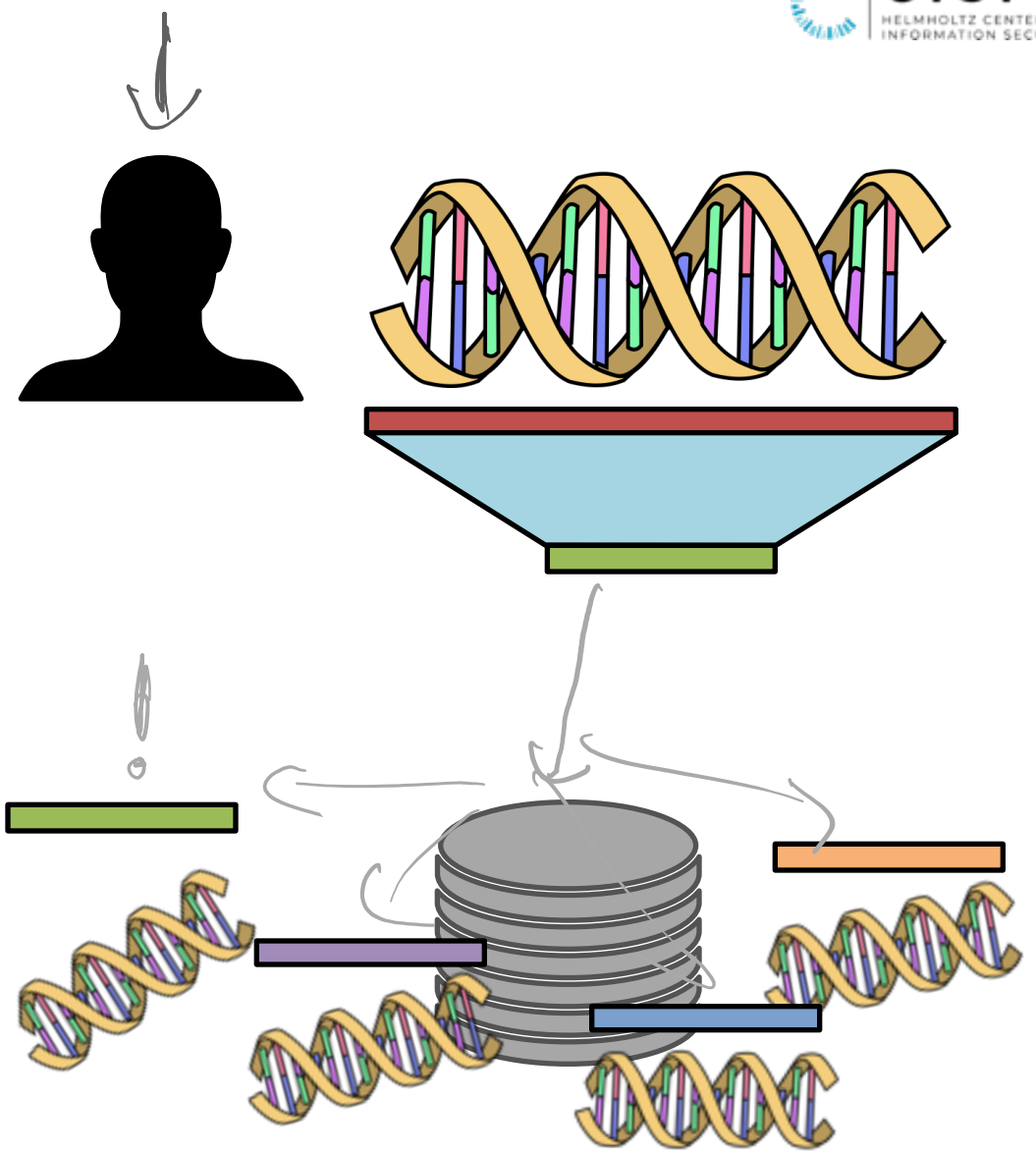
Nico Döttling

Course Topics



Fingerprinting

- Problem: You have a large file/object and want to compare to objects in a database
- E.g. check if you object is in the database
- Object too large to send to the server
- ⇒ Need a small *unique identifier/digest* of your object
- **Hashing!**
- Compress objects into a small fingerprint
- ⇒ Speed up comparisons/membership tests, smaller communication cost



Hash Functions

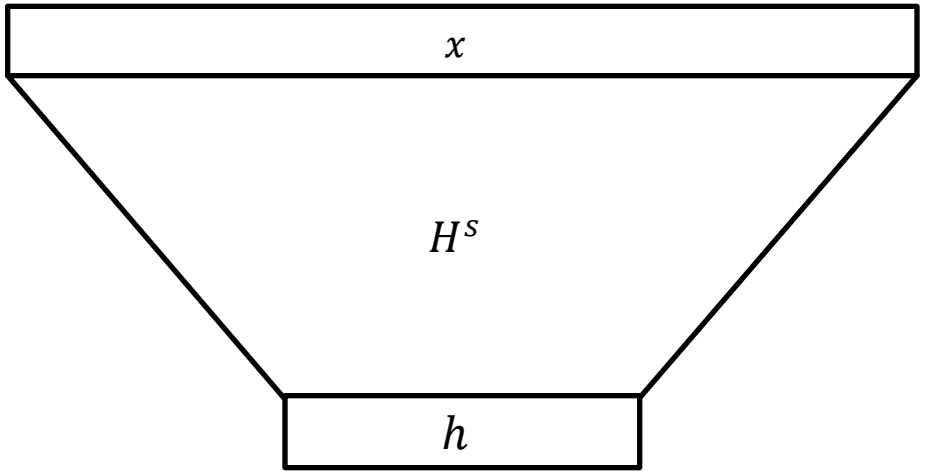
Syntax: A family of hash functions is given by a pair of algorithms (Gen, H)

- $Gen(1^\lambda)$:A probabilistic algorithm which on input 1^λ outputs a key s correctness
- $H^s(x)$: A deterministic algorithm which on input a key s and a string $x \in \{0,1\}^*$ outputs a hash value $h \in \{0,1\}^\ell$

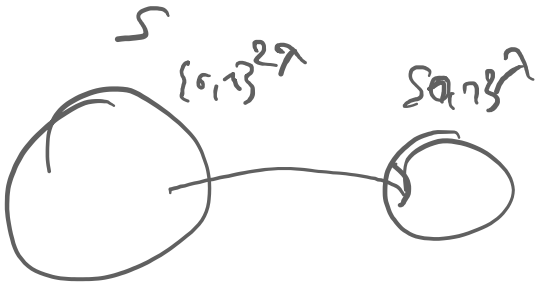
Remarks:

typically $\ell = \lambda$

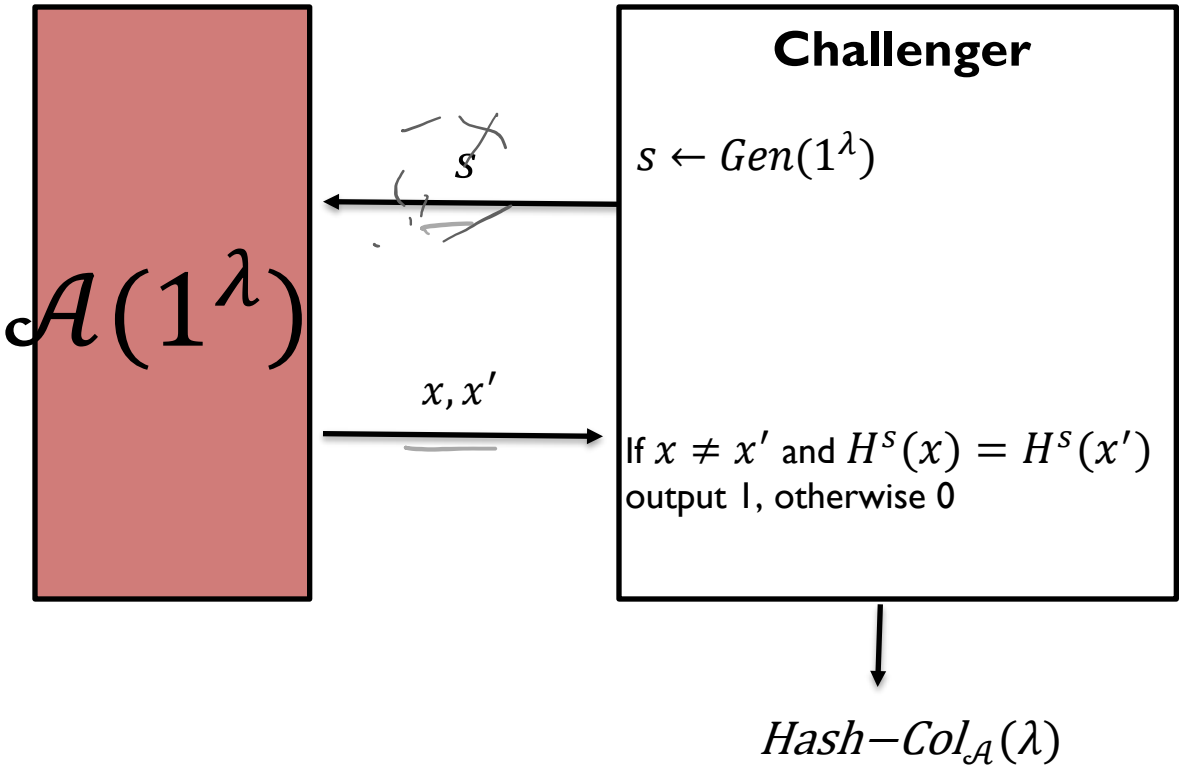
- The output length $\ell = \ell(\lambda)$ only depends on λ
- If H^s is only defined on inputs of length $\ell' > \ell$ we call H a *fixed-length hash function* fixed $H^s: \{0,1\}^{\ell'} \to \{0,1\}^\ell$
- If we don't specify Gen it just chooses a uniformly random $s \leftarrow_\$ \{0,1\}^\lambda$ $s \leftarrow_\$ Gen(1^\lambda)$



Hash Functions: Collision Resistance



$Hash{-}Col_{\mathcal{A}}(\lambda)$ Experiment:



Definition

A hash function (Gen, H) is called **collision-resistant**, if for **every PPT-bounded adversary** $\mathcal{A}$ there exists a negligible function ν s.t. for all $\lambda \in \mathbb{N}$

$$Pr[Hash{-}Col_{\mathcal{A}}(\lambda) = 1] < \nu(\lambda)$$

$H: \mathbb{F}_q^2 \to \mathbb{F}_q$

$H^{\alpha_1,\alpha_2}(x_1,x_2) = \alpha_1 x_1 + \alpha_2 x_2$

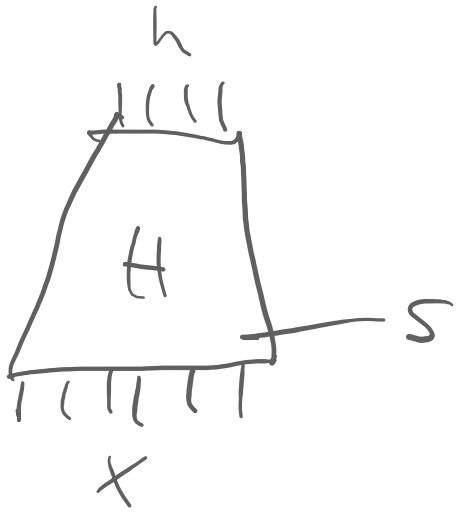
universal hash function $\sim \frac{1}{q}$

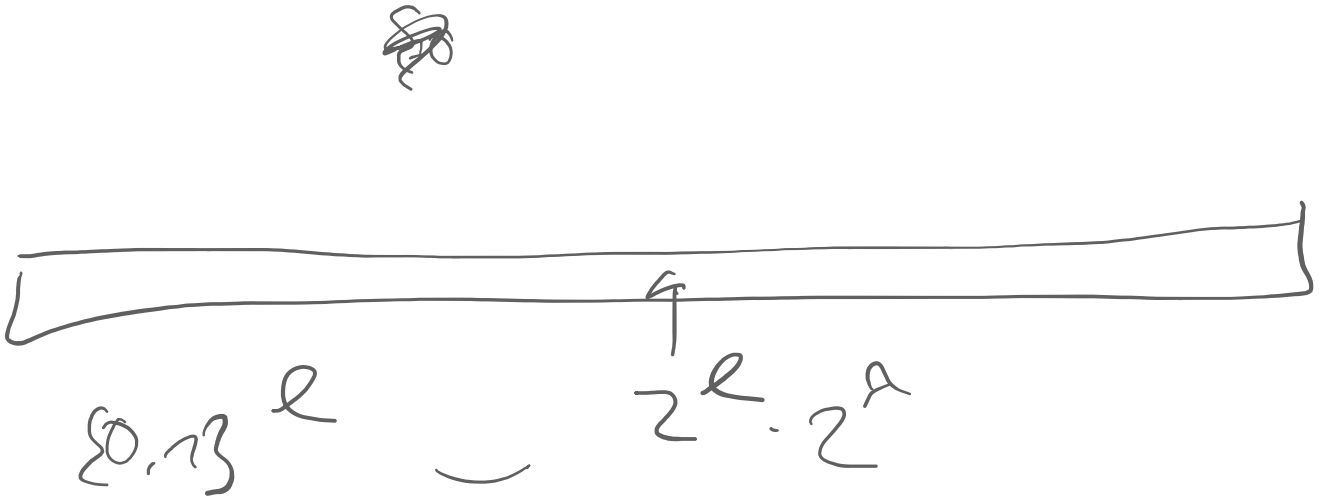
Examples

- Assume H is a collision resistant hash function
- Is H' given by $H'^s(x) = H^s(x \parallel 0^\lambda)$ also collision resistant?
  - Yes!
  - Idea: Collision x, x' for H' yields collision $x \parallel 0^\lambda, x' \parallel 0^\lambda$ for H
- Is H'' given by $H''^s(x_1 \dots x_n) = H^s(x_1 \dots x_{n-1})$ collision resistant?
  - No!
  - $0 \dots 00$ and $0 \dots 01$ hash to the same value

Idealized Hash Functions

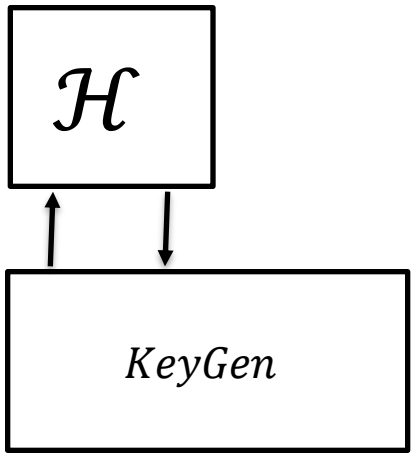
- Sometimes collision resistance isn't enough…
- Sometimes we need hash functions for which it is hard to find arbitrary correlations
- E.g. for a fixed function f it should be hard to find an input x with $H^s(x) = f(x)$
- How should an *ideal* hash function behave?
- Like a random function!

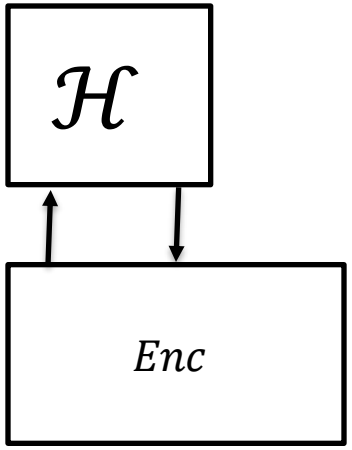


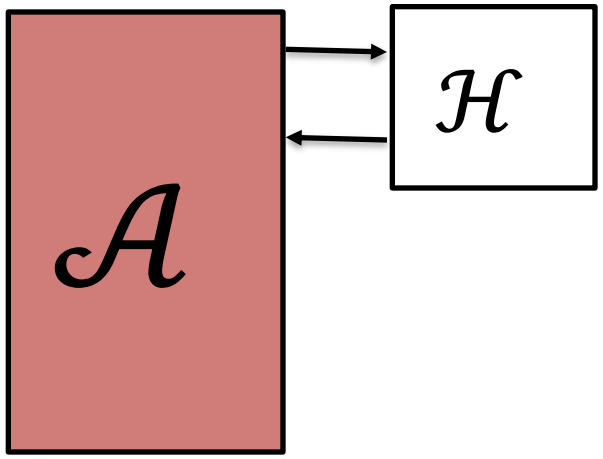


The random oracle model

- Random Oracle (RO): Uniformly random function $\mathcal{H}: \{0,1\}^{\ell} \to \{0,1\}^{\lambda}$ which can only be accessed via oracle-access/blackbox-access
- Random oracle is uniform on positions it has not been queried on
- Models a hash function whose code no-one knows
- Rationale: The only way of learning something useful about a hash function which is modeled by RO is to evaluate it
- To evaluate $\mathcal{H}$ adversary needs to provide input explicitly to random oracle
- Idea: In security proofs reduction controls RO!







The Random Oracle Model

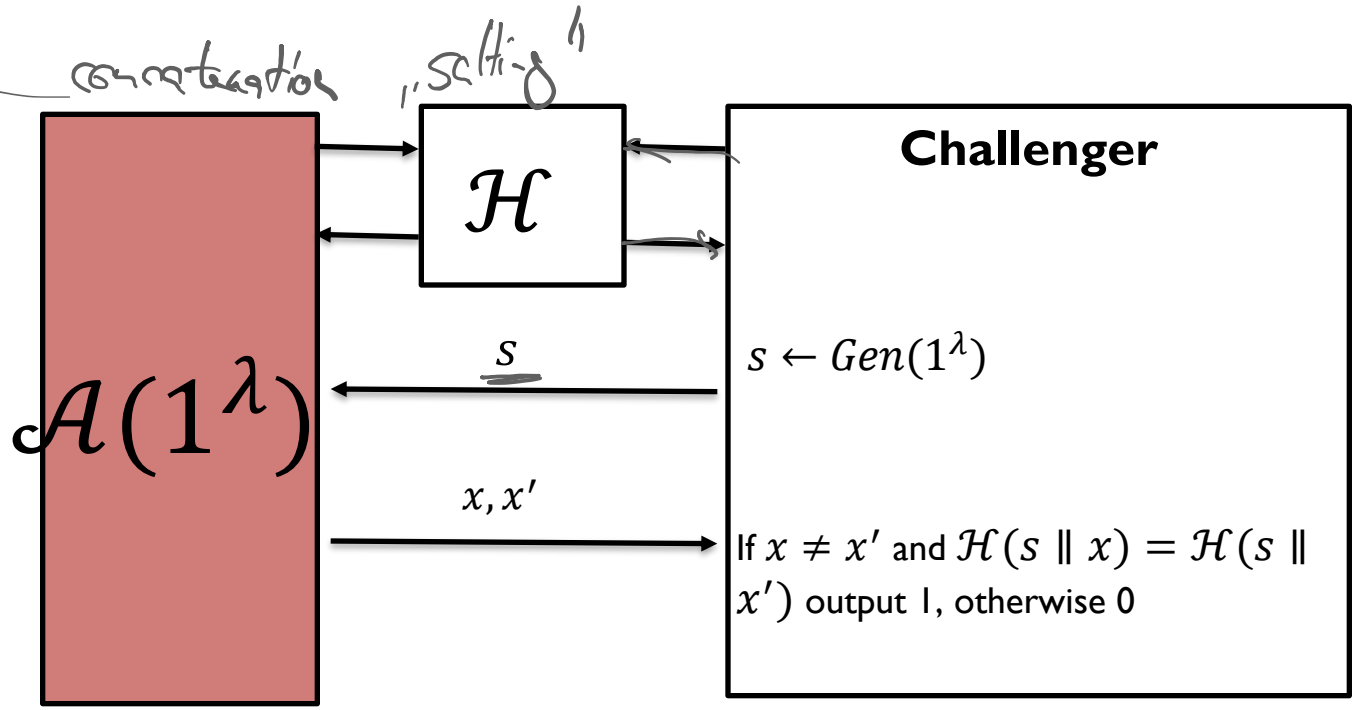
- Obviously, real hash functions are not random oracles
- E.g. real hash functions have small description (e.g. program)
- Sometimes, we only know how to prove constructions secure if $\mathcal{H}$ is modeled as random oracle
- Two step approach:
  1. Model hash function as a RO in the security proof
  2. In the real world, instantiate with actual hash function e.g. SHA3
- Better than no proof at all!
- Proofs in RO model are heuristic
- If a scheme is secure in the RO model, but insecure in real world, adversary must do something non-trivial and interesting with hash function

Example

- Random Oracle yields collision resistant hash function $H^s(x) = \mathcal{H}(s \parallel x)$
- Proofsketch: Assume $\mathcal{A}$ makes at most $q = poly(\lambda)$ queries $x_1, \ldots, x_q$ to $\mathcal{H}$
- $\mathcal{H}$ is uniformly random function, i.e. all function values are uniform and independent
- Probability of single collision: $x_i \neq x_j$

  $Pr[\mathcal{H}(s \parallel x_i) = \mathcal{H}(s \parallel x_j)] = 2^{-\lambda}$

  $i \neq j$
- $Pr[\exists i, j: \mathcal{H}(s \parallel x_i) = \mathcal{H}(s \parallel x_j)] \leq q^2 2^{-\lambda} = negl(\lambda)$ (union bound)
- $\square$

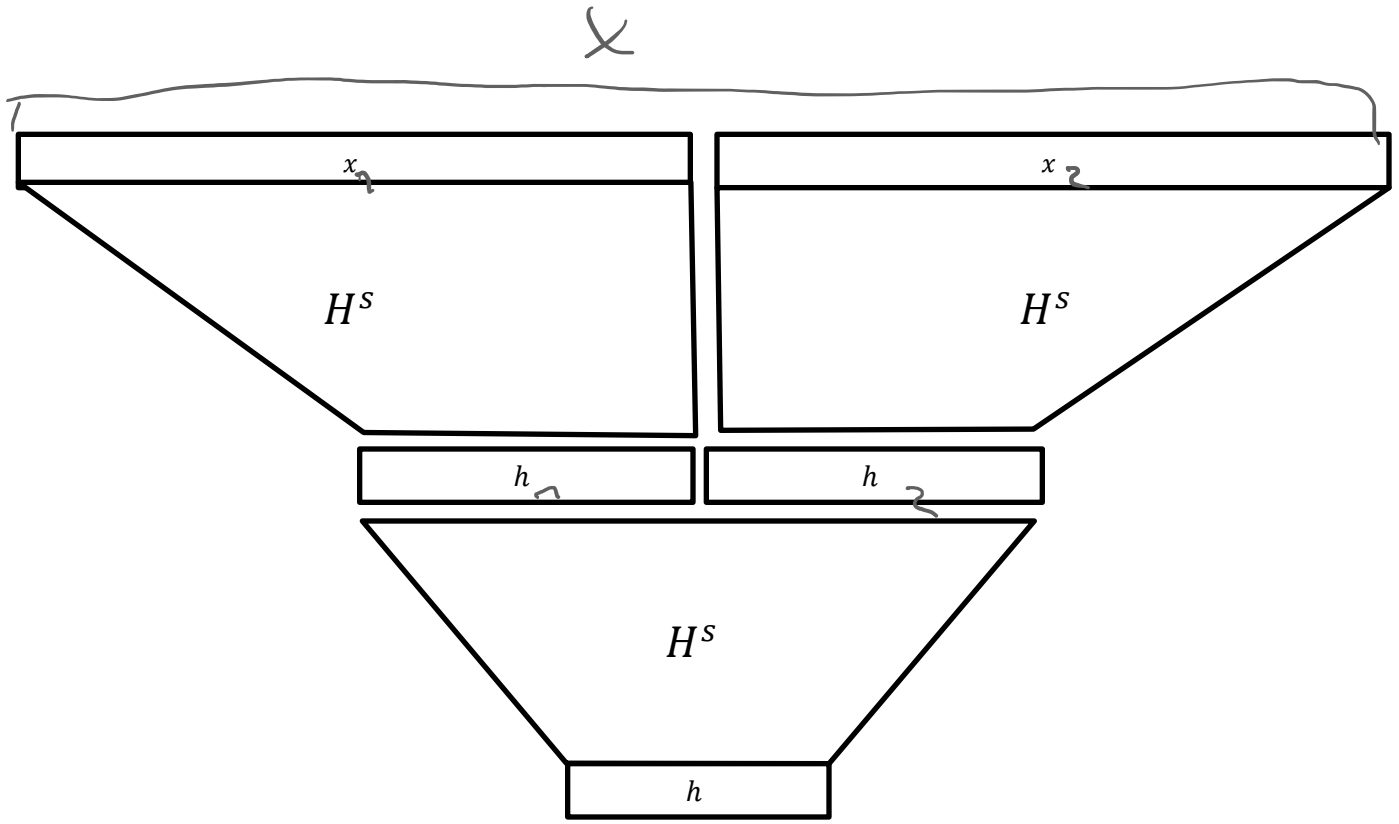


$H: \{0,1\}^{256} \longrightarrow \{0,1\}^{128}$

P/poly ~~bar~~ nonuniform poly

Merkle-Damgård

- Can we extend Hash-Functions?
- Say we have a 2λ to λ bit hash function
- Can we construct a 4λ to λ hash function?



<u>Theorem (Merkle-Damgård, simplified)</u>

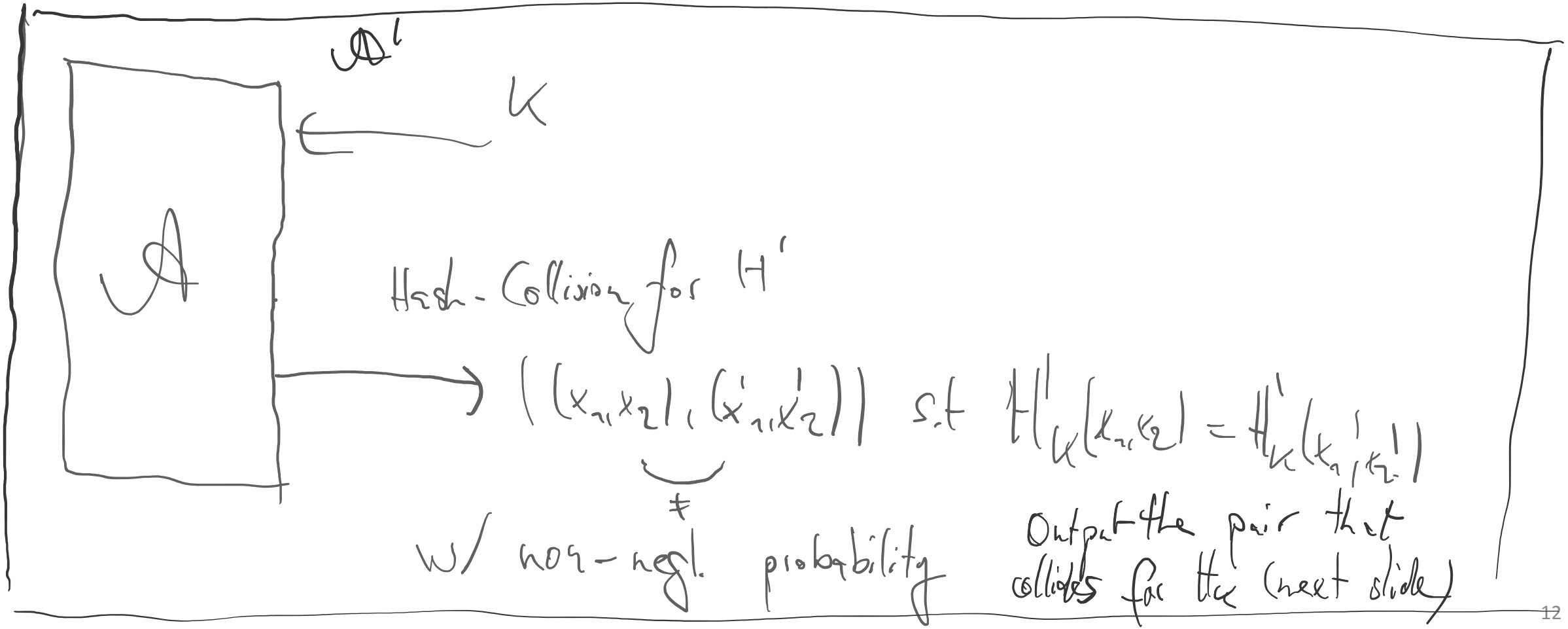
If $H_K: \{0,1\}^{2\lambda} \to \{0,1\}^{\lambda}$ is a collision-resistant hash function, then $H'_K: \{0,1\}^{4\lambda} \to \{0,1\}^{\lambda}$ defined by $H_K(x_1, x_2) = H_K(H_K(x_1) \parallel H_K(x_2))$ is also collision-resistant

concatenation

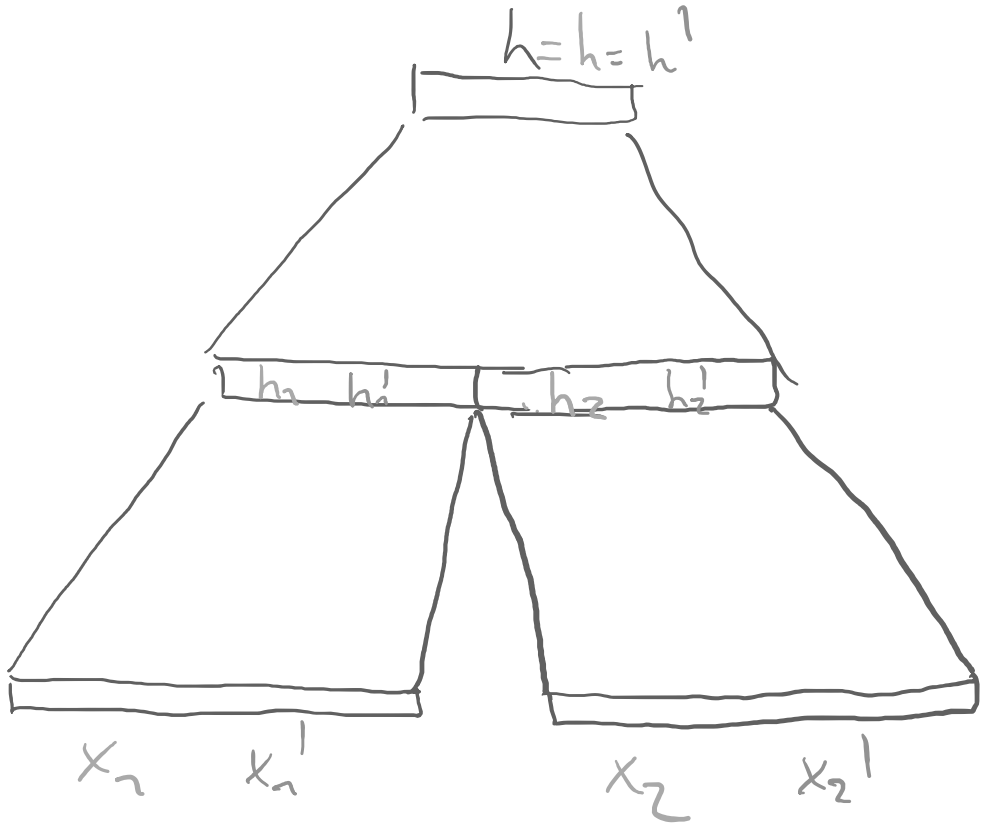
Proof

Assume H' is not col. resistant

$\Rightarrow \exists$ PPT $\mathcal{A}$ s.t.



Proof



Cases:

1. $h_1 = h_1'$ and $h_2 = h_2'$

   If $x_1 \neq x_1'$ then $H_K(x_1) = H_K(x_1')\ (= h_1)$ is a hash-collision for H_K

   otherwise it must hold that $x_2 \neq x_2'$ as $(x_1, x_2) \neq (x_1', x_2')$. Thus $H_K(x_2) = H_K(x_2')\ (= h_2)$ is a hash collision for H_K

2. $h_1 \neq h_1'$ or $h_2 \neq h_2'$

   Then $H_K(h_1 \| h_2) = H_K(h_1' \| h_2')\ (= h)$ is a hash collision for H_K

Summary

- Hash functions compress objects into short digests
- Digests are *computationally* unique
- This is captured in the collision resistance property
- Random oracles model ideal hash functions with stronger properties
- RO model allows for easy proofs!
- Random oracle model is a heuristic: Proofs in the random oracle model don’t necessarily hold in the real world